

VerilogとRISC-Vと検証と あれこれ

テーマごちゃごちゃ

kanataso

2025/11/08 @RTLを語る回(18)



kanataso

- ・ X: [kanapipopipo](#) GitHub: [nananapo](#)
- ・ 修士1年
- ・ RISC-Vシバき人🔥, RTLシバかれ人😓

RISC-VのCPUを書く同人誌を書いています！

- ・ Verylで作るCPU <https://cpu.kanataso.net>

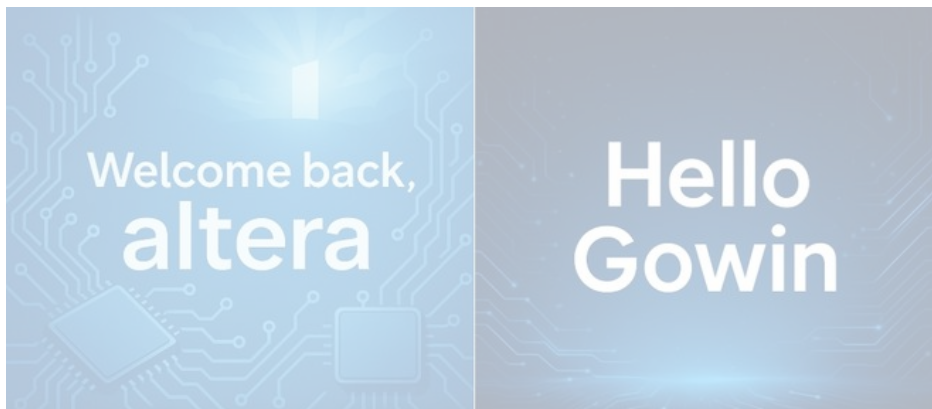
輝かしい称号

- ・ カラオケまねきねこダイヤモンド会員



😡 時間に全く余裕がないので早口発表 😡

積んでいるTangMega138Kに合成しようとしたけど
間に合いませんでした



1. Verylの紹介
2. Verylの導入 (SVの書き換え)
3. RISC-VのCPUのテスト
4. 「Verylで作るCPU」を支える技術

全部独立した話題で四部構成

1. Verylの紹介

- 2. Verylの導入 (SVの書き換え)
- 3. RISC-VのCPUのテスト
- 4. 「Verylで作るCPU」を支える技術

Veryl (ベリル)

新しいハードウェア記述言語 (2022 ~)

- ・ <https://veryl-lang.org>

特徴

- ・ SystemVerilogと同じ抽象度のまま、最適化された構文
 - ・ 型が後置, 末尾カンマ,
- ・ 合成できる、読みやすいSystemVerilogにトランスパイルされる
- ・ SystemVerilogよりコードの行数が減る！！

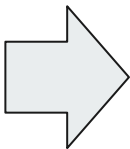
イチオシの機能をご紹介

👍 クロックとリセットの極性の抽象化、省略

- ・ clock, reset型があり、設定(Veryl.toml)で極性を指定できる
- ・ alwaysのクロック、リセットを省略できる

```
module ModuleA (  
    clk: input clock,  
    rst: input reset,  
) {  
    always_ff {  
        if_reset {  
        } else {  
        }  
    }  
}  
[build]  
clock_type = "posedge"  
reset_type = "async_low"
```

veryl build



```
module ModuleA (  
    input var logic clk,  
    input var logic rst  
);  
always_ff @ (posedge clk, negedge rst) begin  
    if (!rst) begin  
    end else begin  
    end  
end  
endmodule
```

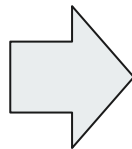
👍 always_ff, always_combの強制、代入方法の限定

- ・ alwaysは存在しない
- ・ always_ffはノンブロッキング代入、always_combはブロッキング代入しか書けない
 - ・ 代入演算子は `=` のみ

```
always_ff @ (posedge clk) begin
    x <= x + 1;
end

always_comb begin
    y = x + 1;
end
```

veryl build



```
always_ff {
    x = x + 1;
}

always_comb {
    y = x + 1;
}
```

この制約はHDL(RTL)初心者が迷いづらくなる？

👍 enumに名前空間が汚されない

列挙型のバリエントは`[enum name]::[variant name]`で指定する
・バリエント名を冗長にする必要が無い

定義

```
typedef enum UIntX {  
    CAUSE_INSTRUCTION_ADDRESS_MISALIGNED,  
    CAUSE_ILLEGAL_INSTRUCTION,  
    CAUSE_BREAKPOINT,  
    CAUSE_LOAD_ADDRESS_MISALIGNED  
    ...
```

規約などにより
prefixをつける



定義

```
enum Cause: UIntX {  
    INSTRUCTION_ADDRESS_MISALIGNED = 0,  
    ILLEGAL_INSTRUCTION = 2,  
    BREAKPOINT = 3,  
    LOAD_ADDRESS_MISALIGNED = 4,  
    ...
```

バリエント名に
prefixが要らない



使用例

```
mcause = CAUSE_INSTRUCTION_ADDRESS_MISALIGNED;
```

使用例

```
mcause = Cause::INSTRUCTION_ADDRESS_MISALIGNED
```

👍 SystemVerilogとの連携

SystemVerilogの要素は`$sv::`で指定して利用する

- ・ SystemVerilogの資産を利用できる 🙌
- ・ `embed (inline) sv{{{}}}`でインラインでSystemVerilogを書くこともできる

```
embed (inline) sv{{{
    package svutil;
        import "DPI-C" context function string get_env_value(input string key);
        function string get_env(input string name);
            return get_env_value(name);
        endfunction
    endpackage
}}}  
package util {  
    function get_env (  
        name: input string,  
    ) -> string {  
        return $sv::svutil::get_env(name);  
    }  
}
```

困ったらSV
を使える



`$sv`を使えば
SVのプロジェクトに
Verylを徐々に導入
できるぞ



👍 ジェネリクス

モジュール、インターフェース、パッケージ、関数、構造体、ユニオンをパラメータ化できる
・かなり強い

// アドレスをデータ単位でのアドレスに変換する

```
function addr_to_memaddr::<DATA_WIDTH: u32, ADDR_WIDTH: u32> (  
    addr: input logic<XLEN>,  
) -> logic<ADDR_WIDTH> {  
    return addr[$clog2(DATA_WIDTH / 8)+:ADDR_WIDTH];  
}
```

```
module plic_memory (  
    clk          : input  clock          ,  
    rst          : input  reset          ,  
    membus       : modport membus_if::<MEMBUS_DATA_WIDTH, XLEN>::slave,  
    irq          : modport irq_if::master ,  
    uart_rx_pending: input  logic        ,  
) {
```

型を変えただけの関数を
定義しなくて良い



インターフェースの
パラメータをポートで
指定できる



👍 modportの簡易な定義

あるmodportをコピー/反転させたmodportや、全部input/outputのようなmodportを簡単に定義できる

- `..input / ..output` : 残りの変数をすべてinput/outputとして追加
- `..same(modport_name)` : modport_nameと同じ変数を同じ方向で追加
- `..converse(modport_name)` : modport_nameと同じ変数を逆の方向で追加

```
interface project_InterfaceA;  
    logic ready;  
    logic valid;
```

```
    modport master (  
        input  ready,  
        output valid  
    );  
    modport slave (  
        output ready,  
        input  valid  
    );  
endinterface
```

masterを変更する
ときにslaveの変更
が必要だな...



```
interface InterfaceA {  
    var ready: logic;  
    var valid: logic;
```

```
    modport master {  
        ready: input ,  
        valid: output,  
    }  
    modport slave {  
        ..converse(master)  
    }  
}
```

スッキリ



👍 関数の引数にmodportを使える

interfaceの変数を引数に書くために、長い長い定義が不要
・簡潔で、interfaceの変数の型の変更に強いコードになる

```
function automatic void assign_device_slave(  
    input  var Device                                device ,  
    output var logic                                req_rvalid,  
    output var logic [MEMBUS_DATA_WIDTH-1:0] req_rdata ,  
    output var core_eei::MemException               req_expt  
);  
    req_rvalid = 1;  
    req_rdata  = 0;  
    case (device) inside  
        Device_RAM : begin  
            req_rvalid = ram_membus.rvalid;  
            req_rdata  = ram_membus.rdata;  
            req_expt   = ram_membus.expt;  
        end
```

書くのが面倒！



```
function assign_device_slave (  
    device: input  Device ,  
    req    : modport Membus::response,  
) {  
    req.rvalid = 1;  
    req.rdata  = 0;  
    case device {  
        Device::RAM : req <- ram_membus;  
        ...  
    }
```

スッキリ



関数内でoutputを割り当てるつもりが無くても、割り当てるように推論されるのが罠かも！

👍 interfaceの接続演算子

interface同士を<>演算子、connect宣言で一気に接続することができる

- ・ ChiselのBulk Connectionのような機能
- ・ (VerylにDecoupled的なやつか継承があると嬉しいと思った)

```
function automatic void assign_device_slave(  
    input var Device          device ,  
    output var logic          req_rvalid ,  
    output var logic [MEMBUS_DATA_WIDTH-1:0] req_rdata ,  
    output var core_eei::MemException req_expt  
);  
    req_rvalid = 1;  
    req_rdata  = 0;  
    case (device) inside  
        Device_RAM : begin  
            req_rvalid = ram_membus.rvalid;  
            req_rdata  = ram_membus.rdata;  
            req_expt   = ram_membus.expt;  
        end
```

書くのが面倒！



```
function assign_device_slave (  
    device: input Device ,  
    req    : modport Membus::response,  
) {  
    req.rvalid = 1;  
    req.rdata  = 0;  
    case device {  
        Device::RAM : req <> ram_membus;  
    ...
```

スッキリ



1. Verylの紹介
- 2. Verylの導入 (SVの書き換え)**
3. RISC-VのCPUのテスト
4. 「Verylで作るCPU」を支える技術

SystemVerilogからVerylへの移行

(10月某日) 発作的な書き換え欲求の発生

- ・ SystemVerilogに嫌気がさした
 - ・ Verylなら少し書くだけなのに....
 - ・ 開発コスト(記述量、認知負荷)を感じた
- ・ プロジェクト全体をVerylにしたい🔥🔥

SystemVerilogからVerylへの移行（失敗）

（10月の連休）書き換え

- ・ 正規表現と手作業とAIでちまちま置き換えていく

SystemVerilogからVerylへの移行（失敗）

（10月の連休）書き換え

- ・ 正規表現と手作業とAIでちまちま置き換えていく

↓

失敗

プロジェクト全体が34000行で、途中で疲れてやめた（敗北）

- ・ 書き換えが思ったよりも面倒だった
- ・ Verylにできないコードの分離が大変

手作業だと思ったよりもつらい

楽だよ

- ちょっとした構文の変更
 - begin, end~ を置き換え
 - <= を = にする (対象がブロッキング代入とノンブロッキング代入を混せていなかった)

苦だよ

- 結構変わった構文の書き換え
 - 型、for文など
- マクロ
 - 今のところVerylにマクロは無い
 - clock付きのassertをマクロにしてるコードはbindに書き換えられた
- 関数がモジュールの外に定義されてる
 - どうすんだ 🤔

トランスパイラが欲しいね...

- ・ あったとして、まずVerylがサポートしないSVの機能を別ファイルに分離する必要がある

The only thing a Big Bang rewrite guarantees is a Big Bang! - Martin Fowler

そもそも一気にプロジェクト全体を書き換えていくのは筋が悪い

- ・書き換え中にバグを埋め込んでしまった場合、炎上が確定する 
- ・**段階的に**置き換えられると良い
 - ・ JavaScript -> TypeScript移行の例に倣う

安易にSVをシバく
べからず

(作業中) 置き換え方法

- ・新しく作るファイルはVerylにする
- ・既存のファイルは、変更するタイミングでVerylに置き換える
 - ・定義している型の名前が元のコードと変わらないように、慎重に書き換えていく

書き換え対象

```
typedef enum logic [2-1:0] {  
    ABC_X,  
    ABC_Y,  
    ABC_Z  
} ABC;
```

同じバリエーション名が出力されるVeryl

```
enum ABC {  
    X,  
    Y,  
    Z,  
}
```

バリエーション名がAbc_XになってしまうVeryl

```
enum Abc {  
    X,  
    Y,  
    Z,  
}
```

1. Verylの紹介
2. Verylの導入 (SVの書き換え)
- 3. RISC-VのCPUのテスト**
4. 「Verylで作るCPU」を支える技術

RISC-VのCPUのテスト

最強デバッグ手法ご紹介: Konataで見る

パイプラインビジュアライザ Konata

・ <https://github.com/shioyadan/Konata>

命令にidをつけることで命令の状態を確認できる

・ おかしの挙動の起点をすぐに発見できる

: 00001a10: bne a5, a0, 0xffffffff0	Rn	Ds	Sc	1	2	Is	Rr	X	Rw	1	2	3	4	5	6
: 00001a00: lbu a1, 0x0(a5)	Dc	Rn	Ds	Sc	Is	Rr	X	Mt	Ma	Rw	1	2	3	4	5
: 00001a04: add a2, a6, a5	Dc	Rn	Ds	Sc	Is	Rr	X	Rw	1	2	3	4	5	6	7
: 00001a08: addi a5, a5, 0x1	Pd	Dc	Rn	Ds	Sc	Is	Rr	X	Rw	1	2	3	4	5	6
: 00001a0c: sb a1, 0x0(a2)	Pd	Dc	Rn	Ds	Sc	1	2	Is	Rr	X	Mt	Ma	Rw	1	2
: 00001a10: bne a5, a0, 0xffffffff0	F	Pd	Dc	Rn	Ds	Sc	Is	Rr	X	Rw	1	2	3	4	5
: 00001a00: lbu a1, 0x0(a5)	Np	F	Pd	Dc	Rn	Ds	Sc	Is	Rr	X	Mt	Ma	Rw	1	2
: 00001a04: add a2, a6, a5	Np	F	Pd	Dc	Rn	Ds	Sc	Is	Rr	X	Rw	1	2	3	4
: 00001a08: addi a5, a5, 0x1	Np	F	Pd	Dc	Rn	Ds	Sc	Is	Rr	X	Rw	1	2	3	4
: 00001a0c: sb a1, 0x0(a2)	Np	F	Pd	Dc	Rn	Ds	Sc	1	2	Is	Rr	X	Mt	Ma	Rw
: 00001a10: bne a5, a0, 0xffffffff0	485): 00001a50: auipc s1, 0x0														
00001a00: lbu a1, 0x0(a5)	486): 00001a54: addi s1, s1, 0x10														
	1): 00001a58: jalr a0, 0x0(s1)														
	): 00001a5c: jalr zero, 0x0(ra)														
	):														
	13	14	15	16	17	F	Pd	Dc	Rn	Ds	Sc	Is	Rr	X	Rw
	13	14	15	16	17	F	Pd	Dc	Rn	Ds	Sc	Is	Rr	X	Rw
							Np	stl	1						

よさそう



Rwステージで
ずっと
ストールしてる



RISC-VのCPUのテスト

RISC-Vのテストスイート、手法たち

- ・ **riscv-tests**

- ・ 基本的な命令や拡張、特権関係の機能を簡単にチェックできる
- ・ 結果確認用のインターフェース(Host/Target InterFace)とCSRの実装が必要

- ・ **RISCOF (riscv-arch-test)**

- ・ riscv-testsよりテストパターンが多いテスト
- ・ CSR不要だがメモリのダンプ機能が必要
- ・ 去年使った記事を書いた (<https://kanataso.hatenablog.jp/entry/2024/12/13/145927>)

- ・ **riscv-isac / riscv-ctg**

- ・ テストを生成するツール (らしい、使ったことない)

- ・ **モデルとの比較**

- ・ 同じISA、同じタイミングで動くモデルが必要なので、単純ではない

tenstorrent/RiESCUE

テnstロレントによるRISC-Vのテストフレームワーク（レスキュー！）

- ・ 2025/9にリリース？
- ・ <https://github.com/tenstorrent/riescue>

RiescueD (Directed Test Framework)

- ・ お手製テストのオペランドをランダム化
- ・ ページにのせてくれたり、OSみたいな環境にのせてくれたり...

RiescueC (Compliance Test Generator)

- ・ アーキテクチャや条件を指定したら、テストを自動生成してくれる
- ・ riscv-coretpというものを使うとテストプラン(シナリオ)を書ける

RiESCUEを入れる

まずはドキュメントに従って導入

依存

- ・ riscv-gnu-toolchain (rv64gc)
- ・ ISSを入れる
 - ・ tenstorrent/whisper
 - ・ tenstorrent/spike

pipでRiESUCUEをインストール

- ・ `pip install git+https://github.com/tenstorrent/riescue.git`

自作コアの設定ファイルを書く

JSONファイルでいろいろ書く

- ・ リセット時のPC
- ・ DRAMのアドレス
- ・ IOのアドレス
- ・ 対応している拡張
- ・ テストの設定

ドキュメントの設定ファイルのサンプルが末尾カンマがついていて不穩

```
{
  "reset_pc": "0x8000_0000",
  "mmap": {
    "dram": {
      "region0": {"address": "0x8000_0000", "size": "0x0100_0000"}
    },
    "io": {
      "htif": {"address": "0x4000_0000", "size": "0x10"}
    }
  },
  "features": {
    "rv64": {"supported": true, "enabled": true, "randomize": 100},
    "i": {"supported": true, "enabled": true, "randomize": 100},
    "m": {"supported": true, "enabled": true, "randomize": 100},
    "a": {"supported": true, "enabled": true, "randomize": 100},
    "f": {"supported": false, "enabled": false, "randomize": 100},
    "d": {"supported": false, "enabled": false, "randomize": 100},
    "c": {"supported": true, "enabled": true, "randomize": 100},
    "h": {"supported": false, "enabled": false, "randomize": 100},
    "v": {"supported": false, "enabled": false, "randomize": 100},
    "u": {"supported": true, "enabled": true, "randomize": 100},
    "s": {"supported": true, "enabled": true, "randomize": 100}
  },
  "test_generation": {
    "secure_access_probability": 30,
    "secure_pt_probability": 0,
    "a_d_bit_randomization": 0,
    "pbmt_ncio_randomization": 0
  }
}
```

RiescueD: テストを作る

コメントでテスト用の情報を記述する
・ランダムに置き換わる値を作れる

test_dataという名前の32ビット値を
マスクによる条件付きで生成

test_dataと0xdeadbeefを足したら
ゼロにならないことを確認する
(test_dataの下位4ビットは0)

```

;#test.author      developer@company.com
;#test.arch        rv64
;#test.priv        machine
;#test.env          bare_metal

.section .code, "ax"

test_setup:
    # Executed before each test, exactly once
    ;#test_passed()

;#random_data(name=test_data, type=bits32, and_mask=0xffffffff)

;#discrete_test(test=test01)
test01:
    li t0, test_data
    li t1, 0xdeadbeef
    add t2, t0, t1

    bne t2, x0, test01_pass
    ;#test_failed()

test01_pass:
    ;#test_passed()
```

RiescueD: テストを実行する

```
riescued --testname [テスト.s] --cpuconfig [設定.json]
```

- ・ --testnameは古いから--testfileしろと出てきて不穩

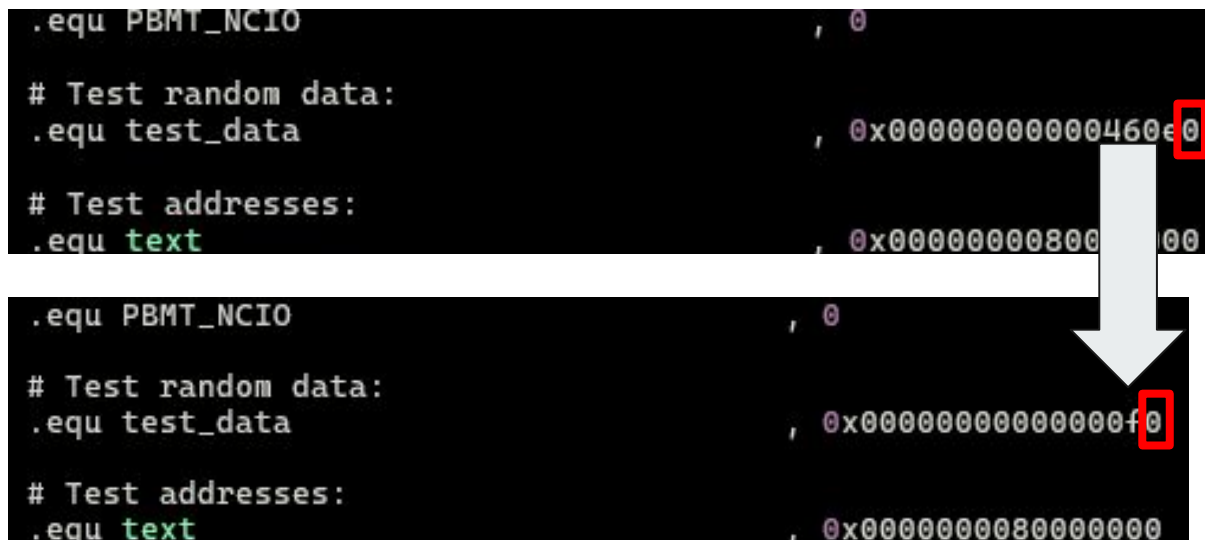
失敗

- ・ なぜかRiESCUEのライブラリのファイルが欠損 - FileNotFoundError
 - ・ GitHubからコピペで持ってくる
- ・ configの記法がドキュメントと変わっていてパースできない
 - ・ パーサが望むように書き換える

RiescueD: テストを実行する

実行できた

- ・ 実行することによってランダムな値に変わっている
- ・ 実行される環境(OS、ベアメタル)別に.incファイルを生成してくれる



```
.equ PBMT_NCIO , 0

# Test random data:
.equ test_data , 0x0000000000004600

# Test addresses:
.equ text , 0x0000000008000000

.equ PBMT_NCIO , 0

# Test random data:
.equ test_data , 0x00000000000000f0

# Test addresses:
.equ text , 0x0000000008000000
```

生成された値が制約（下位4ビットが0）に従っている

RiescueC

BringUpモード

- ・ 設定ファイルでアーキテクチャ、使える(使えない)命令を指定する
- ・ 設定に合わせて、RISC-Vの仕様に違反していないかのテストを作ってくれる

TPモード (右画像)

- ・ テストシナリオをPythonで書ける
- ・ シナリオを動的に変えたら面白そう

spikeのバイナリをtt_spikeとして探そうとするのでオプションでのパス指定が必須😡

```
# Create memory and operations
mem = Memory(size=0x10000)
store_op = Store(memory=mem, offset=0x1000, value=0xDEADBEEF)
load_op = Load(memory=mem, offset=0x1000)

# Create test scenario
scenario = TestScenario([
    mem,
    store_op,
    load_op,
    AssertEqual(src1=load_op, src2=0xDEADBEEF)
])
```

RiESCUE良さそう



- ・ かんたんにランダムな値を扱う命令列のテストを作れる
- ・ 導入コストが低い(はず)
- ・ 命令が配置されるアドレスを変えられるのは強い
 - ・ ページング込みのシナリオを簡単に作れるフレームワークは他にないのでは？

テスト作成のお供としてかなり強力

1. Verylの紹介
2. SystemVerilogからVerylへの書き換え
3. RISC-VのCPUのテスト
- 4. 「Verylで作るCPU」を支える技術**

背景: 段階的に実装していく本を書く

インクリメンタル (ステップバイステップ) に実装する

- ・ だんだんと機能を増やしていく
- ・ 自作する系の本は大体この方式
 - ・ 一気に作るのは難しい / 楽しくない

望む要件 (楽しく続けられる要件)

- ・ すべての段階で成果物をビルドできる
- ・ 一度通ったテストに通り返ける

目次

第I部 RV32I/RV64Iの実装

- 1 環境構築
 - 2 ハードウェア記述言語 Veryl
 - 3 RV32Iの実装
 - 4 Zicsr拡張の実装
 - 5 riscv-testsによるテスト
 - 6 RV64Iの実装
 - 7 CPUのパイプライン化
 - 8 CPUの合成
- 第II部 RV64IMACの実装
- 9 M拡張の実装
 - 10 例外の実装
 - 11 Memory-mapped I/Oの実装
 - 12 A拡張の実装
 - 13 C拡張の実装

<https://cpu.kanataso.net/>

機能が増えていく

実装のコード、原稿中のコードはどう管理している？

課題: 原稿中のコードと実装を対応させ続けるのが難しい

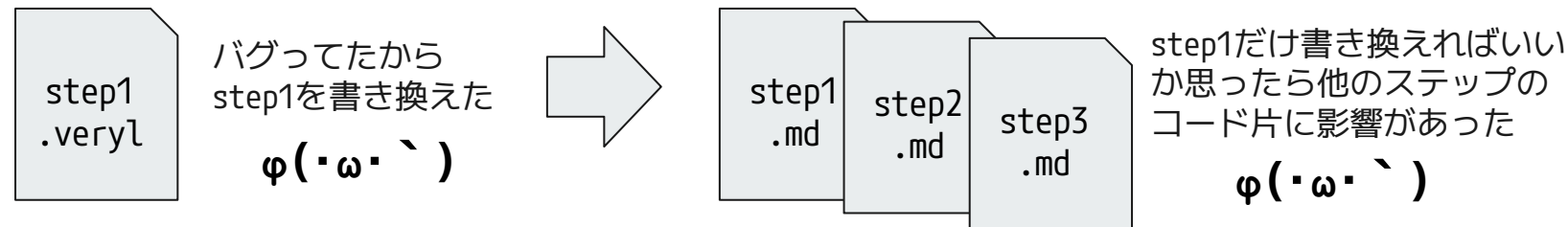
実装しながら執筆するとする

- ・コードの差分を原稿ファイルに転記する



書籍の原稿ファイルがコード変更に非常に弱いものになる

- ・途中でコードを変更したら、コードの差分のコピペし直し



「VeryIで作るCPU」を支える技術

現状: 差分にIDをつけて、自動で原稿に転記

差分にIDを振って、ツール (review-preproc) で原稿ファイルに転記

① 差分

195	195		mode	= PrivMode::M;
196	-		mstatus	= MSTATUS_UXL;
196	+		mstatus	= MSTATUS_SXL MSTATUS_UXL;
197	197		mtvec	= 0;

② 良い感じの範囲にIDをつける

195	+		// #@@range_begin(reset)	
193	196		always_ff {	
194	197		if_reset {	
195	198		mode	= PrivMode::M;
196	-		mstatus	= MSTATUS_SXL MSTATUS_UXL;
199	+		mstatus	= MSTATUS_SXL @{ MSTATUS_UXL};
200	+		// #@@range_end(reset)	
197	201		mtvec	= 0;

あれ？これSXLの方を@で囲むべきでは...?

③ 原稿でIDを指定して自動で転記してもらう

```
74 //list[csrunit.veryl.misamppsxl.reset][mstatus.SXLの初期値を設定する (csrunit.  
75 ## maprange(scripts/23/misamppsxl-range/core/src/csrunit.veryl,reset)  
76 always_ff {  
77     if_reset {  
78         mode = PrivMode::M;  
79         mstatus = @<b>{MSTATUS_SXL | } MSTATUS_UXL;  
80     ## end  
81 }
```

👍 実装変更時の原稿への転記漏れがなくなる
😞 差分にIDをつけるのが非常に面倒くさい

- ・ 通常のコミット、IDをつけるコミット、revertするコミットが必要
- ・ らくらく差分管理、転記手法求む

非常に参考になる記事:

「30 日できる! OS 自作入門」のような段階的に進展するプログラムについての解説書の執筆を Git で支援する

<https://uchan.hateblo.jp/entry/2018/08/11/191600>

Revert "book/23/misamppsxl-range"

nananapo committed on Sep 21

book/23/misamppsxl-range

nananapo committed on Sep 21

implement misa.S, mstatus.SXL

nananapo committed on Sep 21

事例: 新・標準プログラマーズライブラリ RISC-Vで学ぶコンピュータアーキテクチャ 完全入門

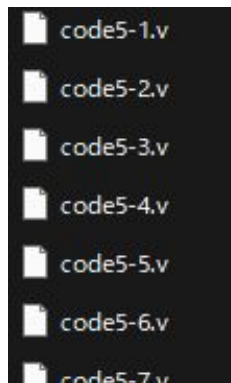
10行くらいの小さなモジュールの組み合わせでCPUを構成する本

- 変更したモジュールについて、モジュール全体のコードを記載、提供
- 細かな差分にIDをつける必要はなさそう

第5章 単一サイクルのプロセッサ

- 5-1 単一サイクルのプロセッサの設計方針
- 5-2 最初の版のプロセッサを設計するための構成要素
- 5-3 add命令を処理するx1のみの単一サイクルのプロセッサ
- 5-4 add命令を処理する単一サイクルのプロセッサ
- 5-5 addとaddi命令を処理する単一サイクルのプロセッサ
- 5-6 add、addi、lw、sw命令を処理する単一サイクルのプロセッサ
- 5-7 add、addi、lw、sw、bne命令を処理する単一サイクルのプロセッサ
- 5-8 シミュレーションの工夫と例題

書籍の実装の全コード?が
提供されている



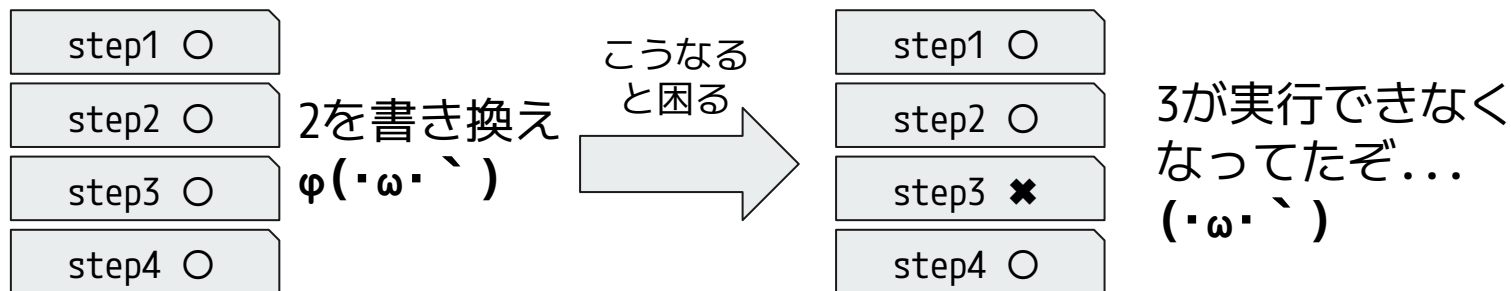
↓この本



すべての段階で実行できるコードを維持する

rebaseしたら、後ろのコミットの段階でビルドできなくなってるかもしれない

- ・書籍の中でバグを見つけたとき、rebaseで修正する



すべての段階で正しく実行できる状態を維持するために、rebaseしたら全コミットのテストを実行しなす

- ・ veril fmtとの差分があるか確認, Verilatorでビルドしてテストを実行

おまけ: GitHubの差分のページに飛ぶボタンをつけてみた

差分のIDにGitのタグを含めて、

`https://github.com/user/repo/compare/コミットID~1..コミットID#diff-ファイル名のSHA256`
を生成するようにした

- ・ 差分の該当ファイル部分に飛べる

▼リスト13.9: ポートの定義 (inst_fetcher.veryl) [差分をみる](#)

```
module inst_fetcher (  
    clk      : input  clock      ,  
    rst      : input  reset      ,  
    core_if: modport core_inst_if::slave,  
    mem_if  : modport Membus::master ,  
){
```

HTML生成ワークフロー

- ① Re:VIEWでMarkdownを生成
- ② Markdownの後処理（上のURL生成、差込）
- ③ VitepressでHTML生成